



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



تمرین ۵

سوال ۲

منبع: اسلاید فصل ۵ بخش ۳ صفحه ۲۲ و ۲۳

غلط است، درواقع به طور خلاصه، در BGP ، اجباری نیست که هر مسیری که یاد می‌گیرید را به دیگران هم یاد بدهید. اگر روتری تصمیم بگیرد مسیری را فیلتر کند ($Drop$ کند) تا ترافیک عبوری ($Transit Traffic$) را مسدود کند، عملاً فرآیند اضافه کردن شناسه خود و ارسال به همسایگان را متوقف کرده است.

درواقع، یک روتر BGP می‌تواند انتخاب کند که شناسه ($Identity$) خودش را به مسیر دریافت شده اضافه نکند و در نتیجه آن مسیر جدید را به همسایگان خود ارسال نکند. دلیل این امر این است که BGP یک پروتکل مسیریابی مبتنی بر سیاست ($Policy - based$) است. این اتفاق می‌تواند در سناریوی زیر رخ دهد: مقصد مسیر دریافت شده یک سیستم خودمختار (AS) دیگر است (نه AS خودِ روتر BGP)، و این روتر BGP تمایلی ندارد که به عنوان یک روتر ترانزیت ($Transit Router$) عمل کند.

در روش‌های $policy - based$ ، برخلاف پروتکل‌هایی مثل $OSPF$ که فقط به دنبال کوتاه‌ترین مسیر هستند، BGP بر اساس سیاست‌های تجاری و فنی مدیر شبکه تصمیم می‌گیرد. یک مدیر شبکه می‌تواند تصمیم بگیرد که ترافیک خاصی را از شبکه خود عبور ندهد.

طبق مثال اسلاید درس هم اگر روتری تصمیم بگیرد که نمی‌خواهد "ترانزیت" باشد، (و نمی‌خواهد که $transit traffic$ را بین ISP ‌های دیگر منتقل کند) وقتی مسیری را از همسایه A دریافت می‌کند که مقصدش همسایه C است، این مسیر را به همسایه B یاد نمی‌دهد ($Advertise$ نمی‌کند)

سوال ۴

۱

برای پیدا کردن این مقادیر، باید کوتاه‌ترین مسیر از Y به بقیه نودها را پیدا کنیم:
به خودش (Y): فاصله ۰ است.

به X : مسیر مستقیم $X - Y$ با هزینه ۶.

به W : مسیر $Y - > X - > W$ با هزینه $4 + 6 = 10$.

به V :

مسیر ۱: $Y - > X - > V$ با هزینه $6 + 6 = 12$

مسیر ۲: $Y - > X - > W - > V$ با هزینه $6 + 4 + 6 = 16$

کوتاه‌ترین: ۱۲.

به U : مسیر ۱: $Y - > X - > V - > U$ با هزینه $2 + 6 + 6 = 14$

کوتاه‌ترین: ۱۴.

پاسخ نهایی قسمت ۱:

ترتیب خواسته شده (u, v, w, x, y) است: 14, 12, 10, 6, 0

۲

در بردار فاصله اولیه (*Initial State*)، هر نود فقط همسایگان مستقیم خود را می‌شناسد و فاصله تا بقیه را بی‌نهایت در نظر می‌گیرد. همسایگان مستقیم U فقط V است. بنابراین فاصله‌های اولیه به این شرح است:
به خودش (U): ۰

به V : همسایه مستقیم است، هزینه ۲.

به W : همسایه مستقیم نیست (x) $\Rightarrow \infty$.

به X : همسایه مستقیم نیست (x) $\Rightarrow \infty$.

به Y : همسایه مستقیم نیست (x) $\Rightarrow \infty$.

ترتیب خواسته شده (u, v, w, x, y) این گونه است: 0, 2, x , x , x

۳

طبق اسلاید فصل ۵ بخش ۲ صفحه ۱۹، مشکل *count to infinity* رخ می‌دهد، یعنی ممکن است تا بی‌نهایت یا تعداد مراحل زیادی، هزینه‌های لینک‌ها آپدیت شوند.

سوال ۶

۱

بدیهی‌ست که در نهایت طبق فرمول بلمن مورد خواهیم داشت:

$$D_x(w) = 2, D_x(y) = 4$$

در واقع برای رسیدن به y باید از w عبور کنیم و نه مسیر مستقیم از x به y .
حال برای محاسبه $D_x(u)$ داریم:

$$D_x(u) = \min\{c(x, w) + D_w(u), c(x, y) + D_y(u)\} = \min\{2 + 5, 5 + 6\} = 7$$

برای قبلی‌ها هم داشتیم:

$$D_x(w) = \min\{c(x, w), c(x, y) + D_y(w)\} = \min\{2, 5 + 2\} = 2$$

۹

$$D_x(y) = \min\{c(x, y), c(x, w) + D_w(y)\} = \min\{5, 2 + 2\} = 4$$

۲

تغییر در $c(x, y)$:

- ابتدا حالتی را در نظر بگیرید که $c(x, y)$ تغییر کند. اگر $c(x, y)$ بزرگ‌تر یا کوچک‌تر شود (تا زمانی که $c(x, y) \geq 1$ باشد)، مسیر با کمترین هزینه از x به u همچنان هزینه‌ای حداقل برابر با ۷ خواهد داشت. بنابراین، تغییر در $c(x, y)$ (در صورتی که $c(x, y) \geq 1$) باعث نخواهد شد که x همسایگان خود را از هیچ‌گونه تغییری مطلع سازد.
- حال اگر هزینه لینک $x - y$ به مقداری کمتر از ۱ کاهش یابد (مثلاً $d = c(x, y) < 1$)، مسیر جدید از طریق y (با هزینه $d + 6$) از مسیر فعلی (که ۷ است) کمتر می‌شود. در این صورت x مسیر جدید را انتخاب کرده و به همسایگان اطلاع می‌دهد.

تغییر در $c(x, w)$:

- اگر $c(x, w)$ به مقداری کمتر یا مساوی ۱ کاهش یابد (مثلاً $\epsilon \leq 1$)، هزینه کل مسیر جدید می‌شود $5 + \epsilon$ که از ۷ کمتر است. پس x هزینه جدید را به همسایگان اطلاع می‌دهد.
- اگر $c(x, w)$ به مقداری بیشتر از ۶ افزایش یابد (مثلاً $d > 6$)، هزینه مسیر از طریق w می‌شود $d + 5$ که بیشتر از ۱۱ خواهد بود. توجه داشته باشید که با افزایش $c(x, w)$ دیگر مسیر $D_x(y)$ برابر ۴ نیست و بیشتر می‌شود تا وقتی که به $D_x(y) = c(x, y) = 5$ می‌رسد. وقتی که بیشتر از ۶ شود. در این حالت، مسیر از طریق y به‌صرفه‌تر می‌شود. پس x سوئیچ می‌کند روی y و هزینه جدید که ۱۱ است را اطلاع می‌دهد.

۳

درواقع هر تغییری در هزینه لینک $c(x, y)$ (و تا زمانی که $c(x, y) \geq 1$ باشد) باعث نخواهد شد که x همسایگان خود را از یک مسیر جدید با کمترین هزینه به u مطلع سازد.

۸

جدول ۱: جدول اجرای الگوریتم دایجسترا (سوال ۸)

$D(z), p(z)$	$D(y), p(y)$	$D(w), p(w)$	$D(v), p(v)$	$D(u), p(u)$	$D(t), p(t)$	N'	Step
$x, ۸$	$x, ۶$	$x, ۶$	$x, ۳$	∞	∞	x	۰
$x, ۸$	$x, ۶$	$x, ۶$	$x, ۳$	$v, ۶$	$v, ۷$	xv	۱
$x, ۸$	$x, ۶$	$x, ۶$	$x, ۳$	$v, ۶$	$v, ۷$	xvu	۲
$x, ۸$	$x, ۶$	$x, ۶$	$x, ۳$	$v, ۶$	$v, ۷$	$xvuw$	۳
$x, ۸$	$x, ۶$	$x, ۶$	$x, ۳$	$v, ۶$	$v, ۷$	$xvuwyt$	۴
$x, ۸$	$x, ۶$	$x, ۶$	$x, ۳$	$v, ۶$	$v, ۷$	$xvuwytz$	۵
$x, ۸$	$x, ۶$	$x, ۶$	$x, ۳$	$v, ۶$	$v, ۷$	$xvuwytz$	۶

خب برای حل این سوال باید از الگوریتم دایکسترا استفاده کنیم. توجه داشته باشید که $D(n)$ کمترین فاصله فعلی در الگوریتم از x تا نود n است و $p(n)$ هم نود قبلی n در این مسیر است. هر بار از میان نود های موجود در صف اولویت، نودی که کوتاه ترین فاصله از x را دارد، انتخاب میکنیم. در صورتی که به همسایگانش مسیری دارد که از مسیر قبلی کمتر است، $D(n), p(n)$ آن نود همسایه را آپدیت میکنیم و جایگاه آن در صف اولویت را هم آپدیت میکنیم. برای بار اول، همسایه های x در صف اولویت هستند. اولین نود انتخابی که کمترین مسیر مستقیم را تا x دارد، v است با فاصله ۳ از x . بعد از آن، هر ۳ی نود های u و y و w کمترین فاصله با فاصله ۶ را از x دارند. اینجا رندم یکی را انتخاب کردیم مثلاً u در مرحله بعد w و در مرحله بعدش هم y را که همان فاصله ۶ را داشتند انتخاب کردیم و در مرحله بعد نود t با فاصله ۷ را انتخاب کردیم و در آخر نود z با فاصله ۸ در صف اولویت را.

۱۰

خیر

از آنجا که Z قصد دارد ترافیک Y را عبور دهد، ناچار است که تبلیغات مسیر (*Route Advertisements*) را به Y ارسال کند. درواقع، برای اینکه Y بتواند ترافیک خود را به Z بفرستد، Z باید به Y بگوید: “من راه رسیدن به این مقاصد (مثلاً کل اینترنت) را بلدم”. این کار با ارسال *Route Advertisement* از طرف Z به Y انجام می‌شود. بدین ترتیب، زمانی که Y بسته‌ای داشته باشد که مقصد آن از طریق Z قابل دسترسی است، Y این گزینه را خواهد داشت که بسته را از طریق Z ارسال کند.

با این حال، مشکل اینجاست که اگر Z مسیرها را به Y تبلیغ کند، Y قادر است آن مسیرها را مجدداً به X تبلیغ کند (*Re-advvertise*). چون روتر Z کنترلی بر رفتار Y ندارد. در واقع به محض اینکه Y یاد گرفت که “برای رسیدن به اینترنت می‌توانم از Z استفاده کنم”، این مسیر را در جدول مسیریابی خود ذخیره می‌کند. حالا Y با X همسایه است. اگر Y طبق سیاست‌های خود تصمیم بگیرد مسیری که از Z یاد گرفته را به X هم یاد بدهد (*Re-advvertise*)، روتر Z نمی‌تواند جلوی این کار را بگیرد بنابراین، در این حالت Z هیچ کنترلی ندارد و نمی‌تواند مانع از عبور ترافیک X از طریق خود شود (زیرا X ترافیک را به Y می‌فرستد و Y آن را به سمت Z هدایت می‌کند). درواقع روتر X پیامی دریافت می‌کند که: “اگر می‌خواهی به اینترنت برسی، بفرست به Y ”. روتر Y می‌داند: “اگر بسته‌ای گرفتم که باید به اینترنت برود، می‌فرستم به Z ”. بنابراین ترافیک X مسیر X به Y به Z را طی می‌کند. در مرتب کردن و ادیت کردن این پاسخ‌ها از این چت استفاده شده است [۱]

۱۱

خروجی که گرفتیم از کد `starter_code_p1.py` چون اینترنت بین الملل قطع بود اینطوری شد:

```
(base) alireza@alirezas-Air starter_code % sudo python3 starter_code_p1.py
Pinging www.time.ir:185.192.112.122 using Python:
```

```
Reply from 185.192.112.122: time=9.97 ms
Reply from 185.192.112.122: time=10.71 ms
Request timed out.
Reply from 185.192.112.122: time=11.28 ms
Reply from 185.192.112.122: time=10.55 ms
Reply from 185.192.112.122: time=12.61 ms
Reply from 185.192.112.122: time=10.33 ms
```

```
(base) alireza@alirezas-Air starter_code % sudo python3 starter_code_p1.py
Pinging www.google.com:216.239.38.120 using Python:
```

```
Reply from 216.239.38.120: time=48.17 ms
Reply from 216.239.38.120: time=46.12 ms
```

```
Request timed out.
Request timed out.
Request timed out.
Request timed out.
Request timed out.
Request timed out.
Request timed out.
```

همونطور که میبینید، برای *time.ir* که داخلی هست، کار میکنه ولی برای *google.com* چون دسترسی به اینترنت بین الملل هنوز برقرار نشده، کار نمیکنه.

توضیحات کد *Ping*

هدف کلی برنامه:

- پیاده‌سازی دستور *Ping* با استفاده از *ICMP Echo Request*
- ارسال بسته‌های *ICMP* به مقصد و دریافت *Echo Reply*
- محاسبه‌ی *Round Trip Time (RTT)*

برای ثابت *ICMP_ECHO_REQUEST*:

- مقدار ۸ نشان‌دهنده نوع *Echo Request* در پروتکل *ICMP* است

برای تابع *checksum*:

- داده‌ها را ۲ بایت ۲ بایت جمع می‌کند
- *carry* ها را *fold* می‌کند
- *ones complement* می‌گیرد
- ترتیب بایت‌ها را *network order* می‌کند

```
thisVal = ord(string[count+1]) * 256 + ord(string[count])
```

برای تبدیل دو بایت متوالی به یک عدد ۱۶ بیتی

```
csum = (csum >> 16) + (csum & 0xffff)
```

جمع بیت‌های *carry* با ۱۶ بیت پایین

```
answer = ~csum & 0xffff
```

محاسبه *ones complement* نهایی

```
answer = answer >> 8 | (answer << 8 & 0xff00)
```

تبدیل ترتیب بایت‌ها به *network byte order*

برای تابع `sendOnePing`:

• ساخت هدر *ICMP* شامل Type, Code, Checksum, ID, Sequence

```
header = struct.pack("bbHHh", ICMP_ECHO_REQUEST, 0, myChecksum, ID, 1)
```

ایجاد هدر اولیه با *checksum* صفر

• قرار دادن زمان ارسال در *payload*

```
data = struct.pack("d", time.time())
```

این زمان برای محاسبه *RTT* استفاده می‌شود

• محاسبه *checksum* نهایی روی *header* و *data*

```
myChecksum = checksum(str(header + data))
```

• ارسال *packet* به مقصد با استفاده از *raw socket*

```
mySocket.sendto(packet, (destAddr, 1))
```

برای تابع `receiveOnePing`:

• انتظار برای دریافت پاسخ با استفاده از *select*

```
whatReady = select.select([mySocket], [], [], timeLeft)
```

جلوگیری از بلاک شدن برنامه

• دریافت *packet* از *socket*

```
recPacket, addr = mySocket.recvfrom(1024)
```

• استخراج *ICMP header* از داخل *IP packet*

```
icmpHeader = recPacket[20:28]
```

برای تابع doOnePing:

- ساخت Raw Socket با پروتکل ICMP

```
mySocket = socket(AF_INET, SOCK_RAW, icmp)
```

- ارسال و دریافت یک Ping کامل

```
sendOnePing(mySocket, destAddr, myID)
delay = receiveOnePing(mySocket, myID, timeout, destAddr)
```

برای تابع ping:

- تبدیل نام میزبان به IP

```
dest = gethostbyname(host)
```

- ارسال Ping به صورت پیوسته با فاصله ۱ ثانیه

```
while 1:
    delay = doOnePing(dest, timeout)
    print(delay)
    time.sleep(1)
```

نکته اجرایی:

- اجرای این برنامه نیازمند دسترسی root می‌باشد و باید با دستور زیر اجرا شود:

```
sudo python3 starter_code_p1.py
```

و این هم خروجی کد بعدی starter_code_p2.py:

```
(base) alireza@alirezas-Air starter_code % sudo python3 starter_code_p2.py
```

```
Password:
```

```
Traceroute to google.com (216.239.38.120)
```

```
1 192.168.1.1 rtt=3.66 ms
1 192.168.1.1 rtt=3.46 ms
2 100.75.128.1 rtt=4.42 ms
2 100.75.128.1 rtt=3.69 ms
3 10.22.29.61 rtt=11.88 ms
3 10.22.29.61 rtt=19.36 ms
4 10.22.29.62 rtt=6.51 ms
```

```

4 10.22.29.62 rtt=6.67 ms
5 10.22.29.65 rtt=15.00 ms
5 10.22.29.65 rtt=9.18 ms
6 10.21.249.202 rtt=18.51 ms
6 10.21.249.202 rtt=19.48 ms
7 * * * Request timed out.
7 * * * Request timed out.
8 * * * Request timed out.
8 * * * Request timed out.
9 * * * Request timed out.
9 * * * Request timed out.
10 * * * Request timed out.
10 * * * Request timed out.

```

حالا اینجا ما به جای ۳ تا، ۲ تا پیغام پروب میفرستیم. همانطور که میبینید از روتر ۷ به بعد تایم اوت میشه. احتمالا چون اینترنت بین الملل قطع است، اینطوری میشه. از طرفی احتمالا میشه این نتیجه رو گرفت که ۶ تا روتر اول در کشور هستن و یه جور داخل شبکه ملی محسوب میشه.

توضیحات کد *Traceroute*

هدف کلی برنامه:

- پیاده‌سازی دستور *Traceroute* با استفاده از *ICMP Echo Request*
- کشف روترهای میانی با افزایش تدریجی مقدار *TTL*
- محاسبه‌ی *RTT* برای هر *hop*

برای ثابت‌ها:

- *ICMP_ECHO_REQUEST* = ۸ برای ارسال *Echo Request*
- *MAX_HOPS* حداکثر تعداد روترهای قابل بررسی
- *TIMEOUT* حداکثر زمان انتظار برای پاسخ
- *TRIES* تعداد تلاش برای هر مقدار *TTL*

برای تابع *build_packet*:

- ساخت یک *ICMP Echo Request* کامل
- قرار دادن *PID* به عنوان *Packet ID* برای یکتایی *packet*

- قرار دادن timestamp در *payload* برای محاسبه *RTT*

```
header = struct.pack("bbHHh", ICMP_ECHO_REQUEST, 0, 0, myID, 1)
```

- ساخت *header* اولیه با *checksum* صفر (موقت)

```
data = struct.pack("d", time.time())
```

- ذخیره زمان ارسال *packet* در *payload*

```
myChecksum = checksum(header + data)
```

- محاسبه *checksum* نهایی روی *header* و *data*

- برای تابع *get_route*:

- تبدیل نام میزبان به *IP* با DNS resolution

- افزایش تدریجی مقدار TTL از ۱ تا MAX_HOPS

```
for ttl in range(1, MAX_HOPS + 1):
```

- هر مقدار *TTL* نشان‌دهنده یک *hop* در مسیر است.

- تنظیم مقدار TTL روی *socket*

```
mySocket.setsockopt(IPPROTO_IP, IP_TTL, struct.pack('I', ttl))
```

- این خط باعث می‌شود *packet* فقط به اندازه *TTL* روتر عبور کند.

- ارسال *packet* و ثبت زمان ارسال

```
mySocket.sendto(packet, (hostname, 0))
```

```
t = time.time()
```

- استفاده از *select* برای جلوگیری از بلاک شدن برنامه

```
ready = select.select([mySocket], [], [], TIMEOUT)
```

- اگر پاسخی دریافت نشود، *hop* به عنوان *timeout* گزارش می‌شود.

برای پردازش پاسخ *ICMP*:

- استخراج *ICMP header* بعد از *IP header* (۲۰ بایت)

```
icmpHeader = recvPacket[20:28]
```

- تشخیص نوع پیام *ICMP*

```
types, code, checksum_rcv, packetID, sequence =  
struct.unpack("bbHHh", icmpHeader)
```

انواع پیام‌های *ICMP*:

- Type 11 (Time Exceeded): روتر میانی که *TTL* در آن صفر شده است
- Type 0 (Echo Reply): رسیدن *packet* به مقصد نهایی
- Type 3 (Destination Unreachable): مقصد یا مسیر غیرقابل دسترس

برای محاسبه *RTT*:

```
rtt = (timeReceived - t) * 1000
```

زمان رفت و برگشت *packet* بر حسب میلی‌ثانیه محاسبه می‌شود.

پایان *Traceroute*:

- در صورت دریافت Echo Reply با کد صفر برنامه مسیر را کامل یافته و متوقف می‌شود.

```
elif types == 0:  
    return
```

کد ها را آپدیت کردم و به همراه فایل پاسخنامه آپلود کردم.

مراجع

[1] “gapgpt chat with gemini 3 pro,” <https://gapgpt.app/share/>, 2025.